TUF(The Update Framework)の紹介

2026/10/22

OpenChain Japan Community Day vol.36

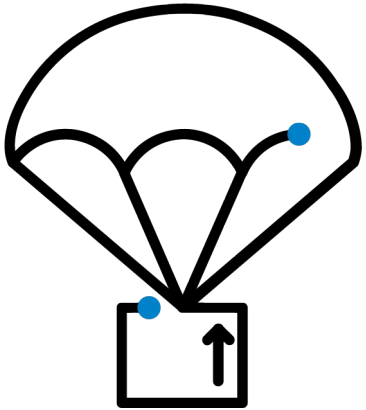
サイバートラスト株式会社

河原 颯太

Copyright Cybertrust Japan Co., Ltd. All rights reserved.

公開

本発表のテーマ：TUF



TUF

TUF

The Update Framework

https://theupdateframework.io/

Index

- TUFの目的・脅威モデル
- TUFにおけるロールと用語整理

- TUFにおける鍵管理とメタデータの信頼
- (Targetsロールによる)targetsの検証
- メタデータの整合性と最新保証

- demo

TUFの目的・脅威モデル

Copyright Cybertrust Japan Co., Ltd. All rights reserved.

TUFの目的



cybertrust

“a framework for securing software update systems”

TUFは「**ソフトウェアアップデートシステムを安全にするためのフレームワーク**」である

“We are not creating a universal update system”
“a simple and flexible way that applications can have high levels of security with their software update systems”

TUFは既存のソフトウェアアップデートシステムを
塗り替えたり統合するものではない
既存のソフトウェアアップデートシステムが自身を,
攻撃から保護するために活用できるメカニズムを提供

TUFでできること・特徴

- 強力な鍵管理メカニズム
  - ロール分割により鍵を分けて管理，被害範囲を小さく
  - 配布物(Targets)に対する柔軟な権限付与・管理
  - Root of Trustを起点とした検証・信頼のチェーンモデル

- 配布物の検証
  - 本物か?
  - 最新か?
  - 矛盾がないか?

TUFでできること・特徴

- 強力な鍵管理メカニズム
  - ロール分割により鍵を分けて管理，被害範囲を小さく
  - 配布物(Targets)に対する柔軟な権限付与・管理
  - Root of Trustを起点とした検証・信頼のチェーンモデル

- 配布物の検証
  - 本物か?
  - 最新か?
  - 矛盾がないか?

TUFの脅威モデル

- 中身のすり替え: Arbitary installation, Wrong software installation, Extraneous dependencies
- バージョン操作: Rollback, Fast-forward, Indefinite freeze, Mix-and-match
- 可用性・資源の妨害: Endless data, Malicious mirrors preventing update
- 鍵の侵害: Vulnerability to key compromises

https://theupdateframework.github.io/specification/latest/#goals-to-protect-against-specific-attacks より

TUFにおけるロールと用語整理

Copyright Cybertrust Japan Co., Ltd. All rights reserved.

公開

用語整理

- Targets
  - 実際に配布したいアップデートやファイルの総称
- TUFリポジトリ
  - メタデータとTargetsを配布する場所
- TUFクライアント
  - TUFの検証ステップを実施して信頼できるアップデートを取得するクライアント
- Root of Trust
  - ユーザーが信頼の起点として用いる，任意のバージョンのroot.json

TUFにおける4つのロール

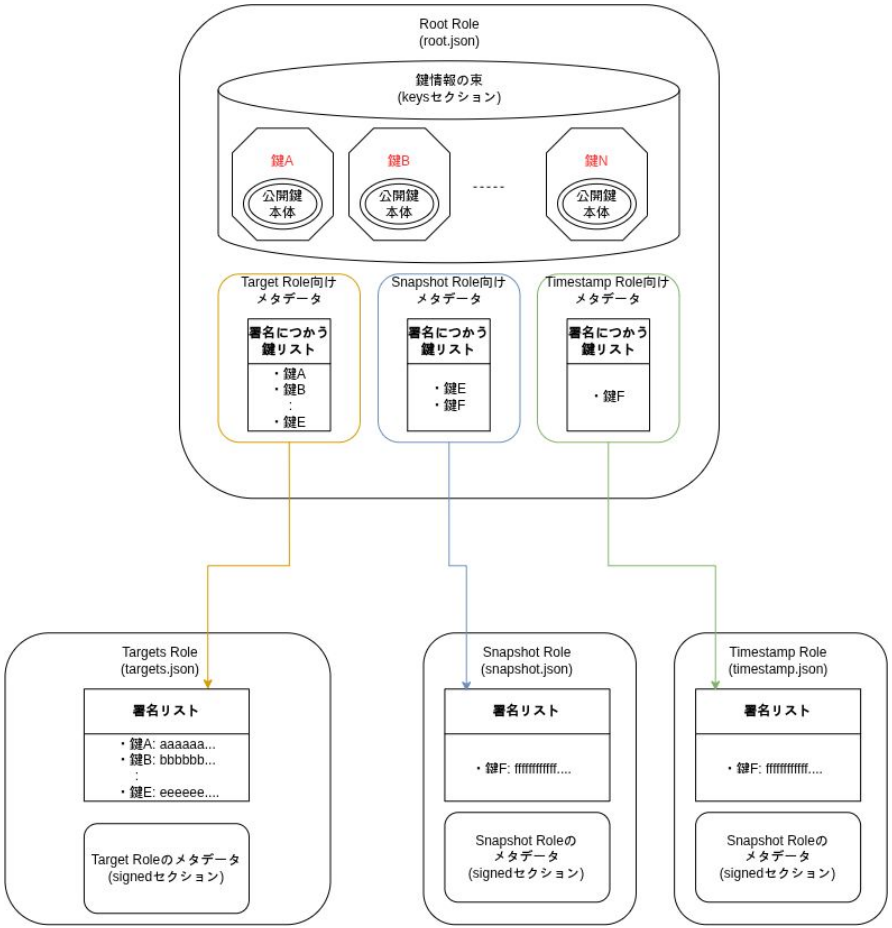
- TUFにおける4つのロール(詳細後述)
  - Root Role
  - Targets Role
  - Snapshot Role
  - Timestamp Role
- アップデートの配布者は，各ロールに対応するメタデータを配布する
  - 例: 1.root.json (<version>.<role>.json)
  - ユーザーはTUFリポジトリからメタデータを取得・検証してアップデートを取得できる
  - メタデータは署名付きフォーマット(DSSEではない独自フォーマット)

TUFにおける鍵管理

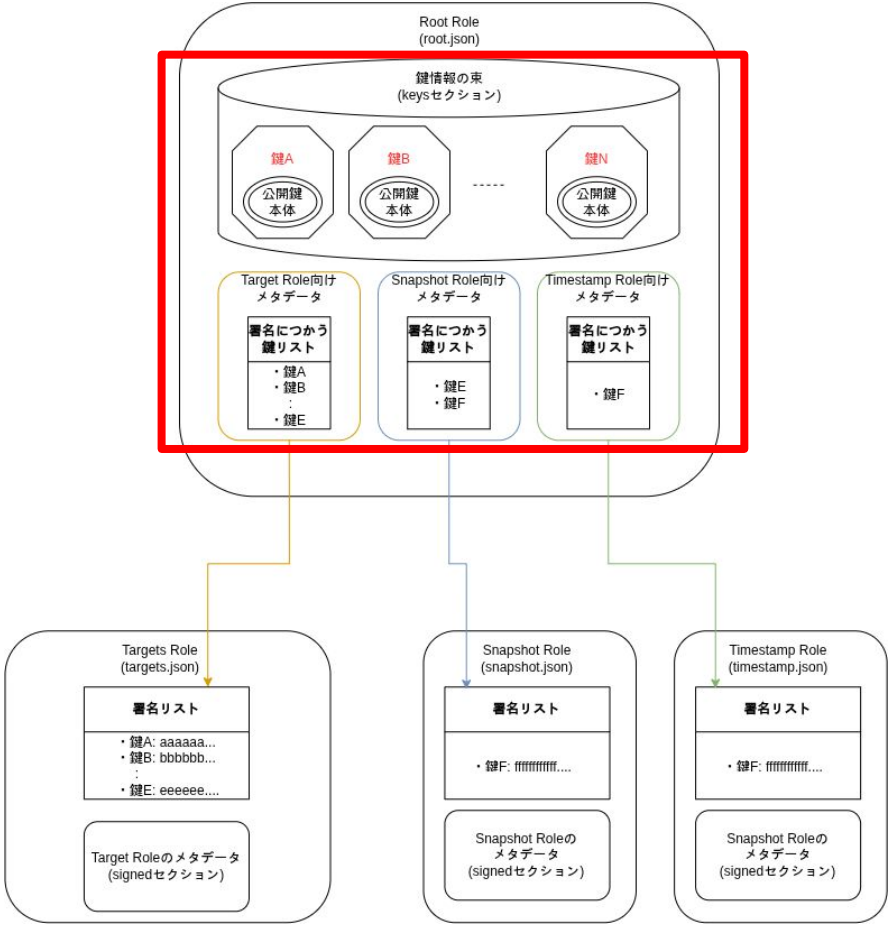
Copyright Cybertrust Japan Co., Ltd. All rights reserved.

公開

root.jsonのデータとほかのメタデータの関係

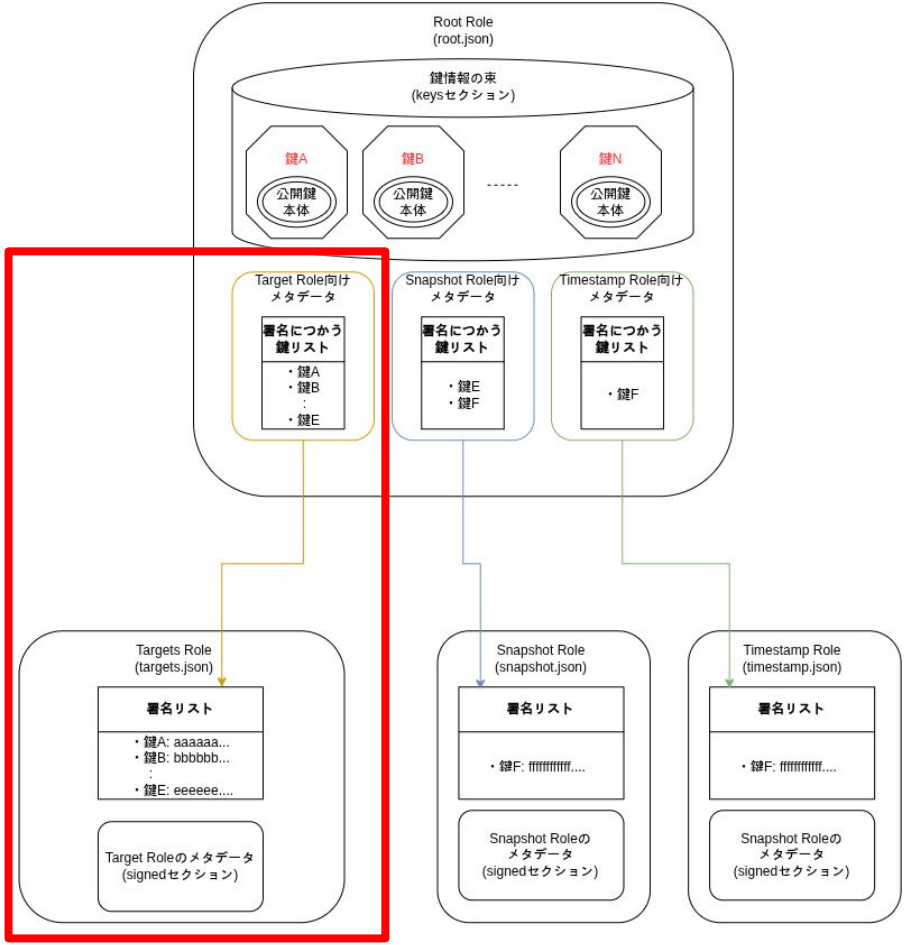


root.jsonのデータとほかのメタデータの関係



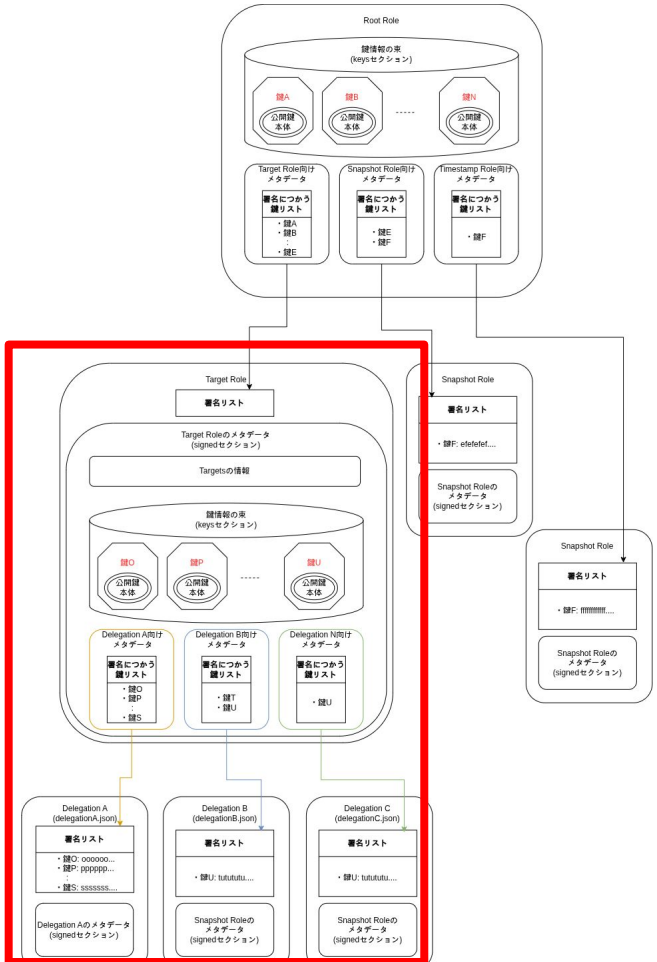
- root.jsonに含まれるデータ
  - 各メタデータの署名に使われたキーペアの公開鍵そのもの
  - 各ロールで使われる署名鍵の宣言(リスト)

root.jsonのデータとほかのメタデータの関係

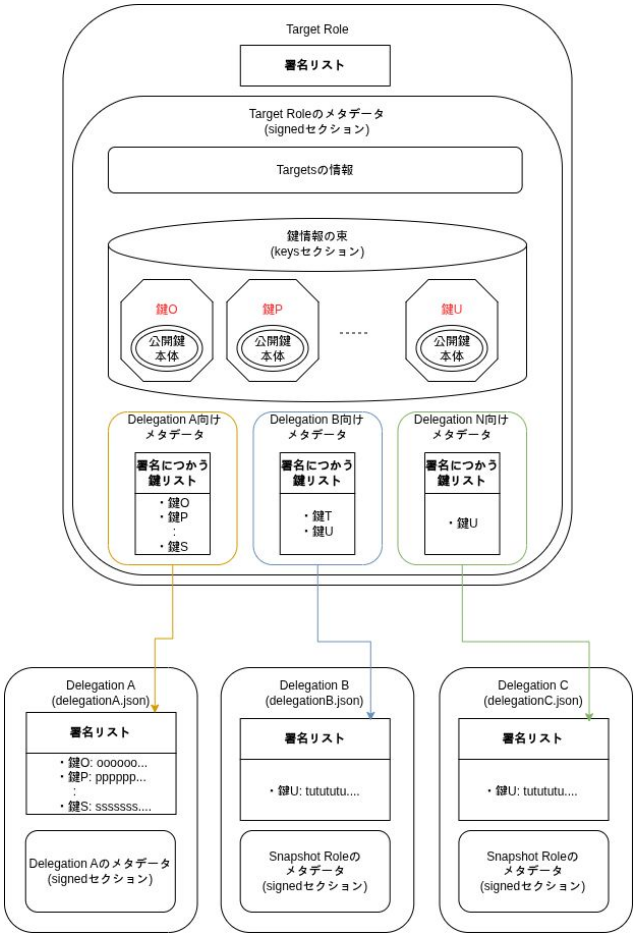


- targets.jsonに付与される署名は…
  - root.jsonのTarget Role向け鍵リストの公開鍵で検証できる
  - 署名は複数付与できる
  - (しきい値を設定可能(図では割愛))

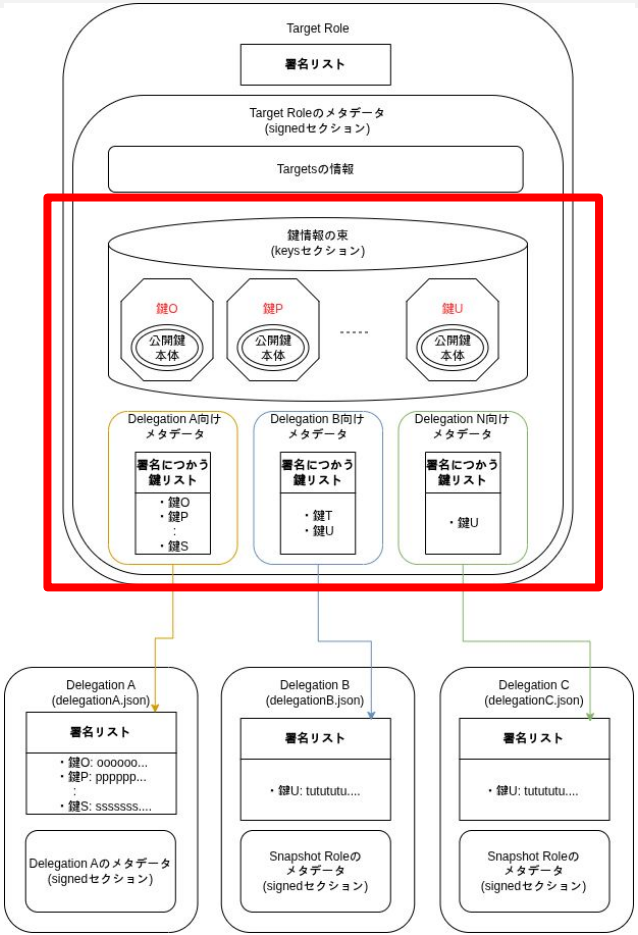
delegationの補足



delegationの補足

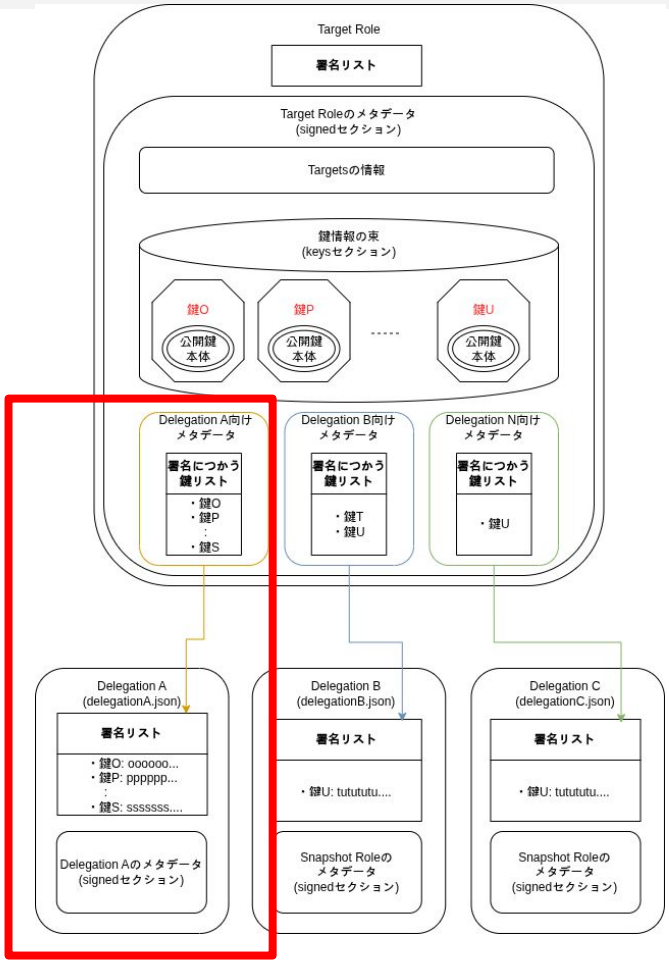


delegationの補足



- targets.jsonに含まれるデータ
  - 各delegationの署名に使われたキーペアの公開鍵そのもの
  - 各delegationで使われる署名鍵の宣言(リスト)

delegationの補足



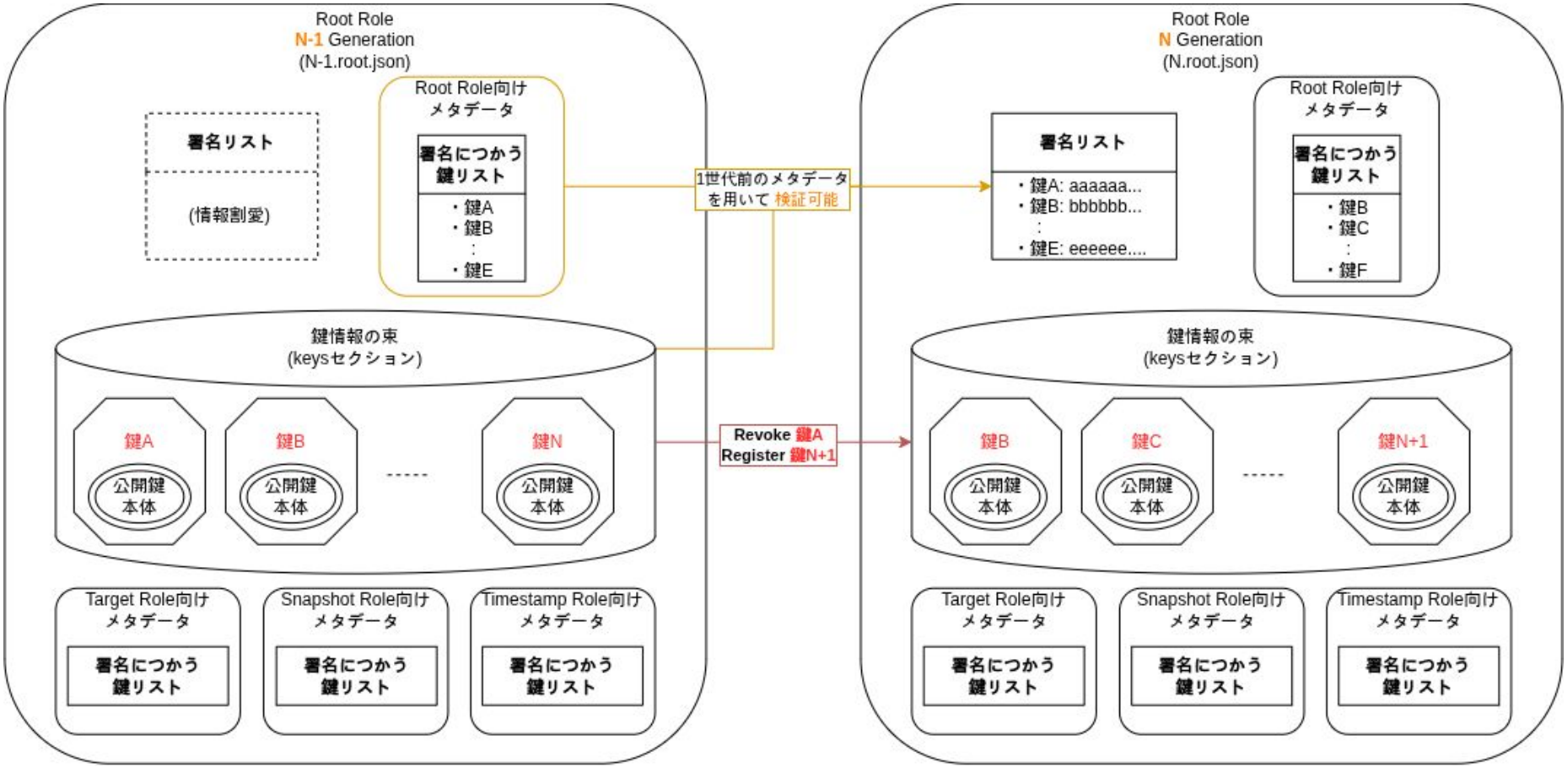
- Delegation Aに付与される署名は…
  - targets.jsonのDelegation A向け鍵リストの公開鍵で検証できる
  - 署名は複数付与できる
  - （しきい値は割愛）

メタデータの信頼

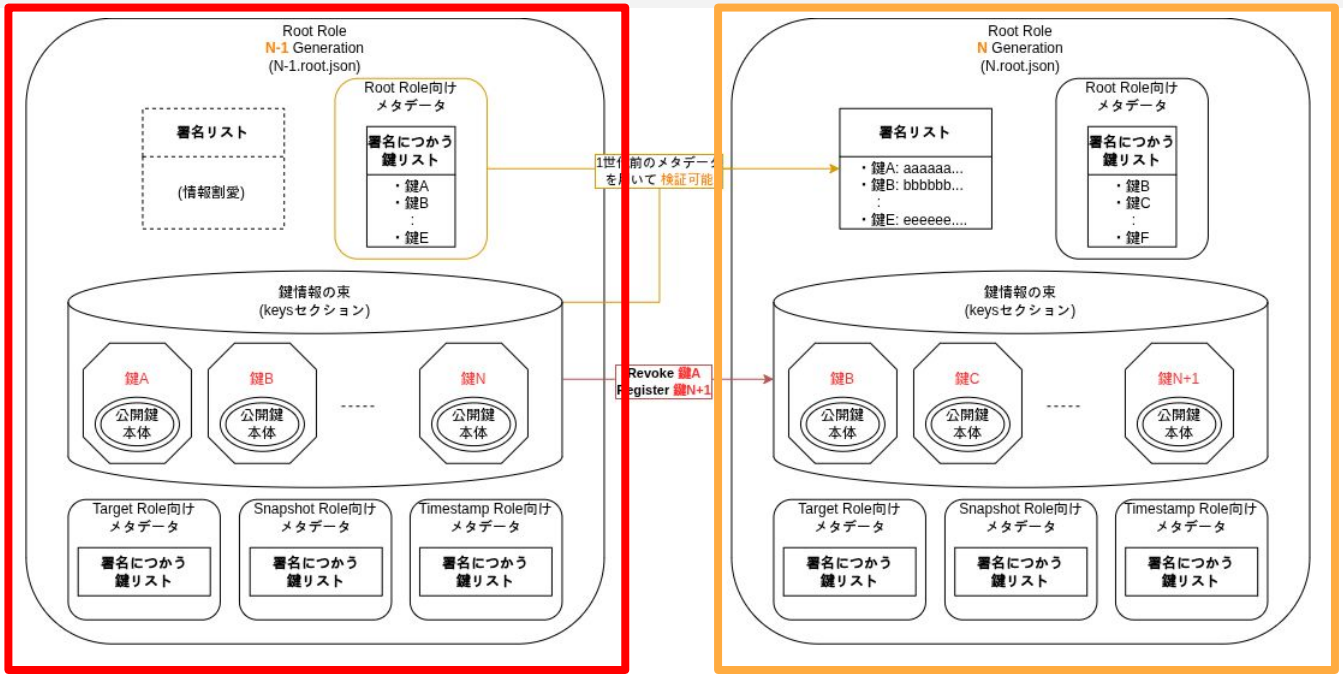
Copyright Cybertrust Japan Co., Ltd. All rights reserved.

公開

Root Roleの連続性



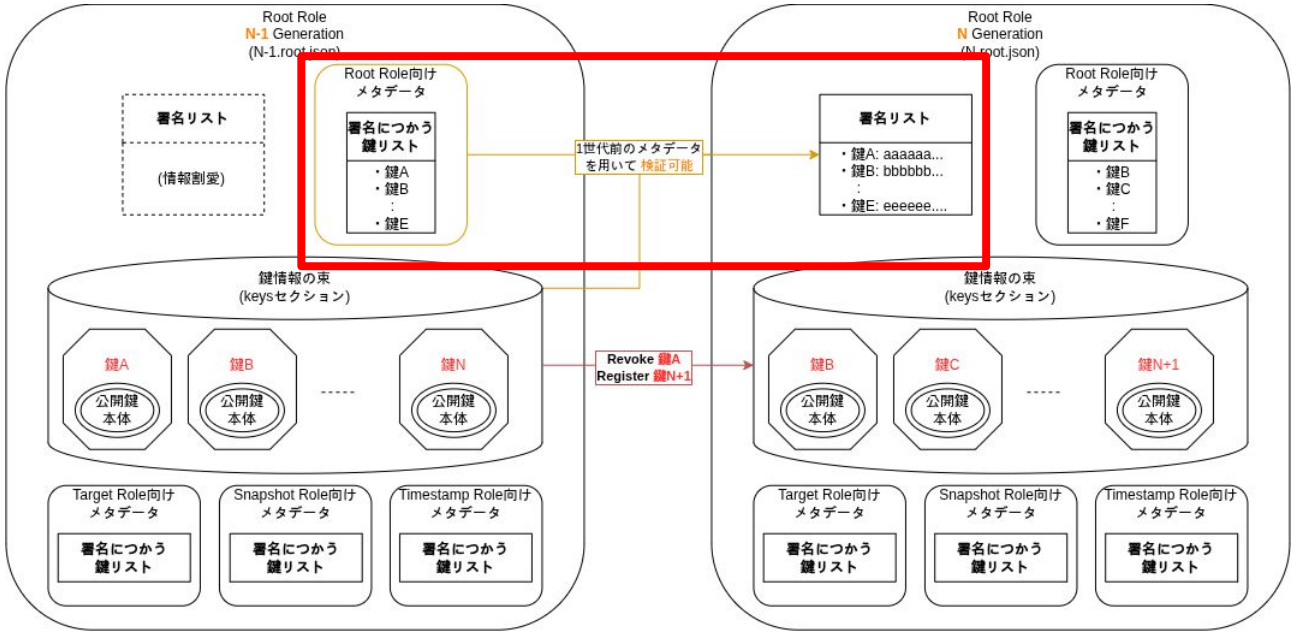
Root Roleの連続性



ある特定の時期に作成した
バージョン‘N-1’
の root.json(N-1.root.json)

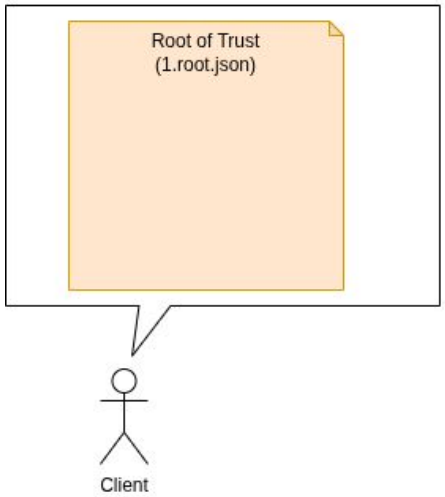
その後のアップデートで作成した
バージョン‘N’
の root.json(N.root.json)

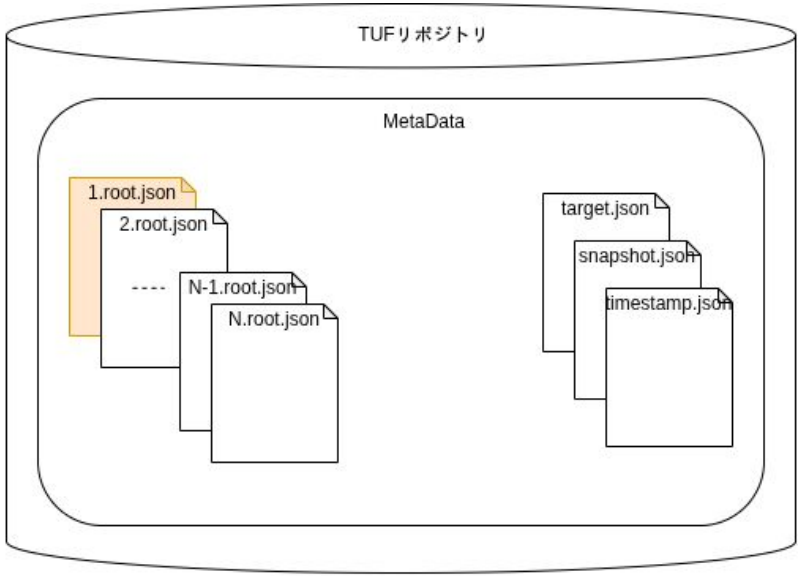
Root Roleの連続性



- root.json内には，「Root Role向けの署名鍵リスト」も存在
- N.root.jsonの署名に使われる鍵は，N−1.root.jsonにリスト
- N−1.root.jsonの情報でN.root.jsonを検証することができる

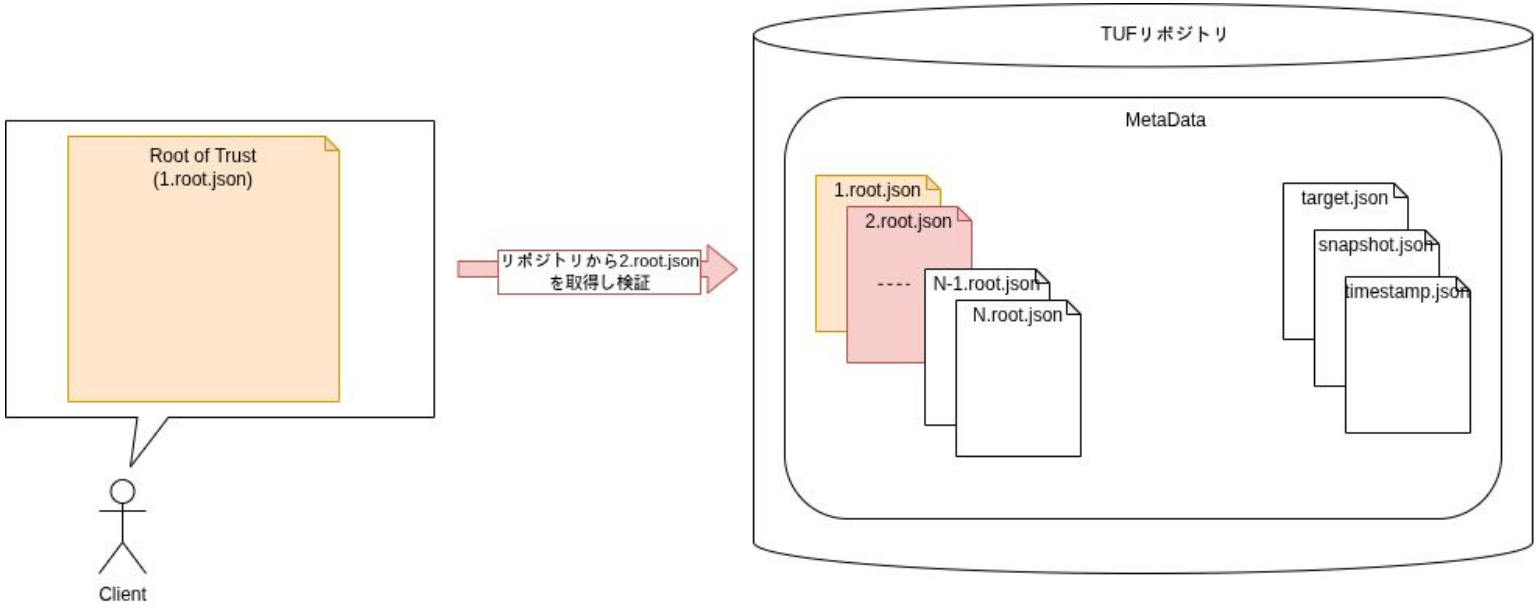
Root of Trustを起点としたメタデータの信頼





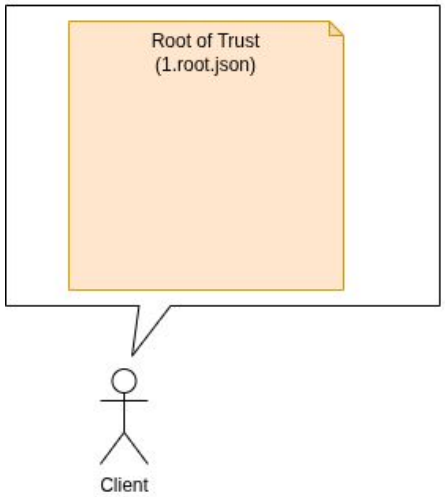
- ユーザーは “Root of Trust(信頼の起点)”を，TUF外の安全な方法で事前に取得しておく
- バージョンはなんでもよく，最新でなくて良いことが重要

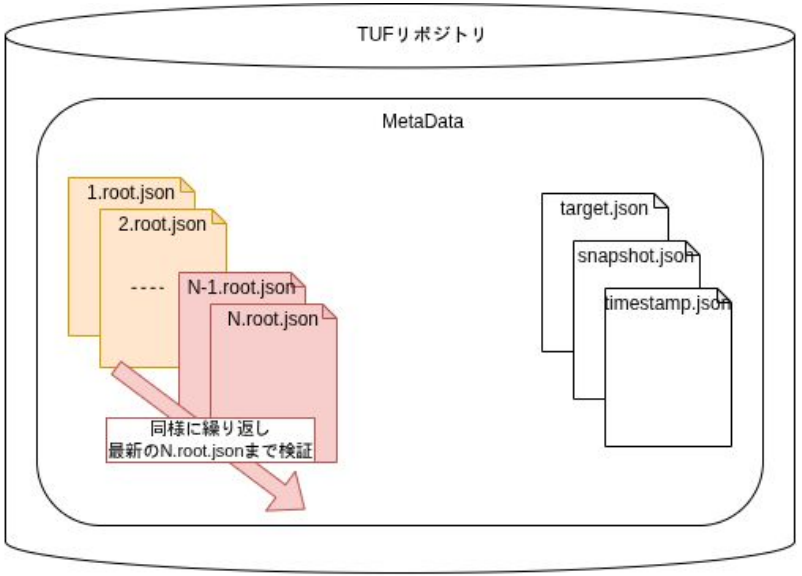
Root Roleの連続性



- 2.root.jsonを取得し，1.root.jsonのデータだけで検証・信頼することが可能

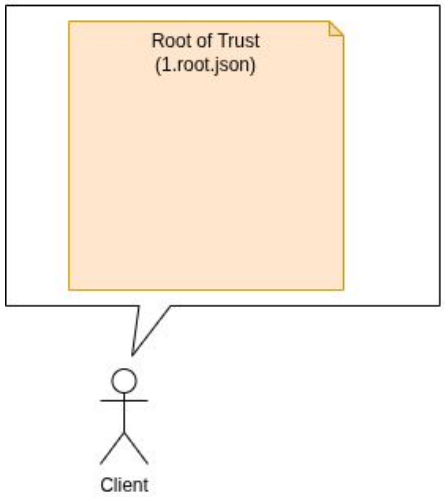
Root Roleの連続性

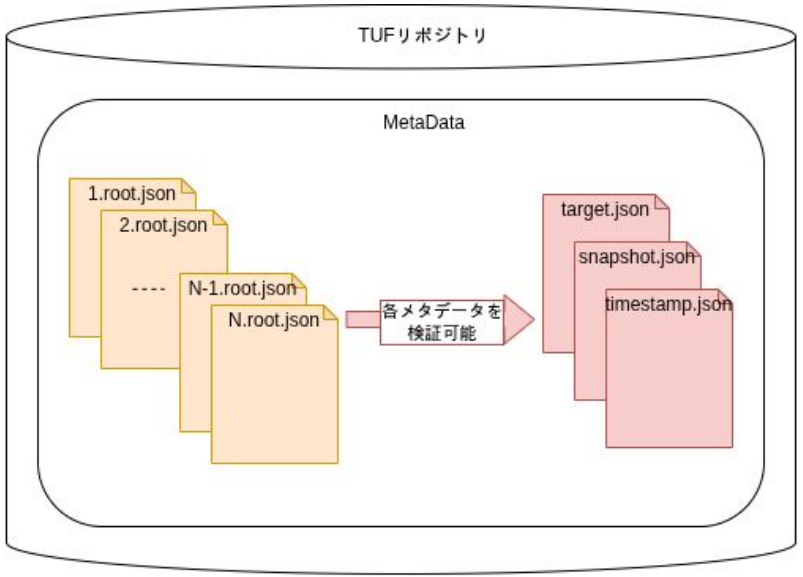




- これを繰り返し，最新のN.root.jsonまで検証・信頼することが可能

Root Roleの連続性



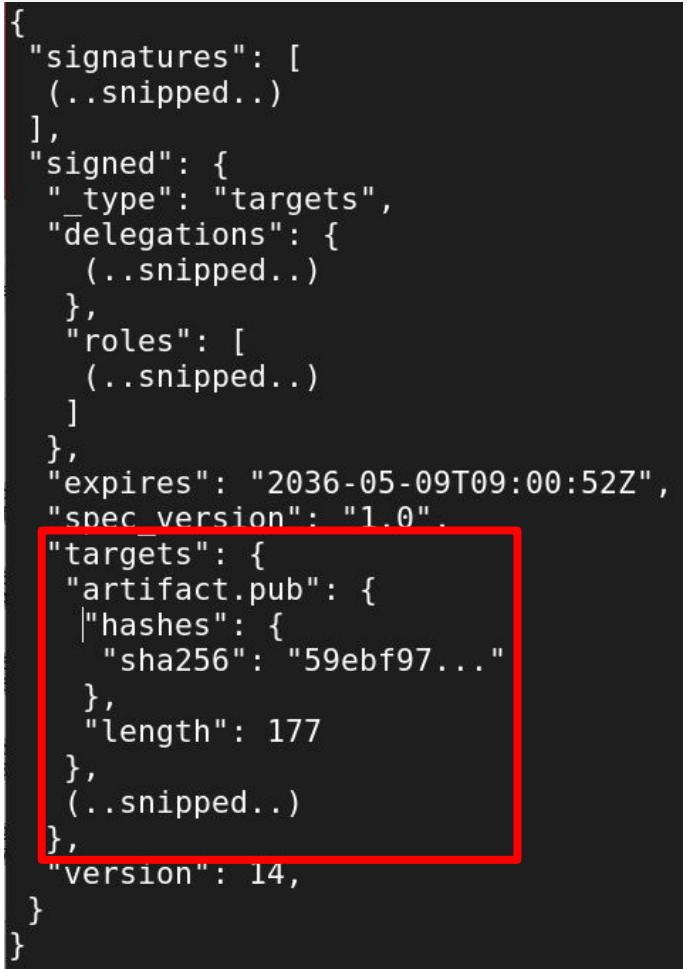


- N.root.jsonから，最新のRoot Role以外の各メタデータを検証・信頼することが可能

Targetsの検証

Copyright Cybertrust Japan Co., Ltd. All rights reserved.

targets.jsonの例とtargetsの検証



```
{
 "signatures": [
  (..snipped..)
 ],
 "signed": {
  "_type": "targets",
  "delegations": {
    (..snipped..)
   },
   "roles": [
    (..snipped..)
   ]
  },
  "expires": "2036-05-09T09:00:52Z",
  "spec_version": "1.0",
  "targets": {
   "artifact.pub": {
    "hashes": {
     "sha256": "59ebf97..."
    },
    "length": 177
   },
   (..snipped..)
  },
  "version": 14,
 }
}
```

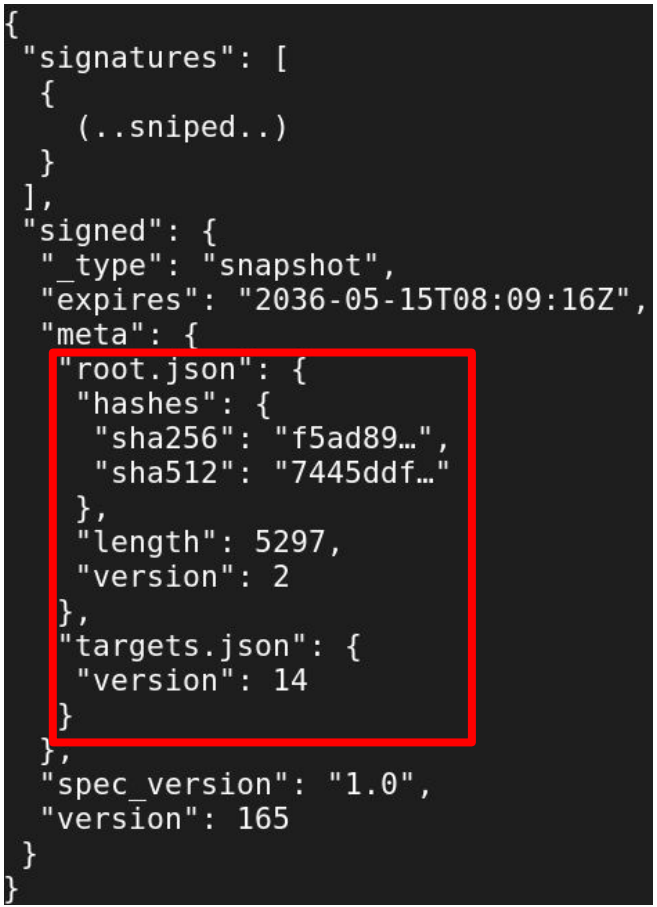
- targetsのハッシュ値やファイル長が各ファイルごとに示される
- TUFクライアントはここに示されるハッシュ値やサイズを実際のファイルのものと比較して検証できる
-

メタデータの整合性と最新性

Copyright Cybertrust Japan Co., Ltd. All rights reserved.

公開

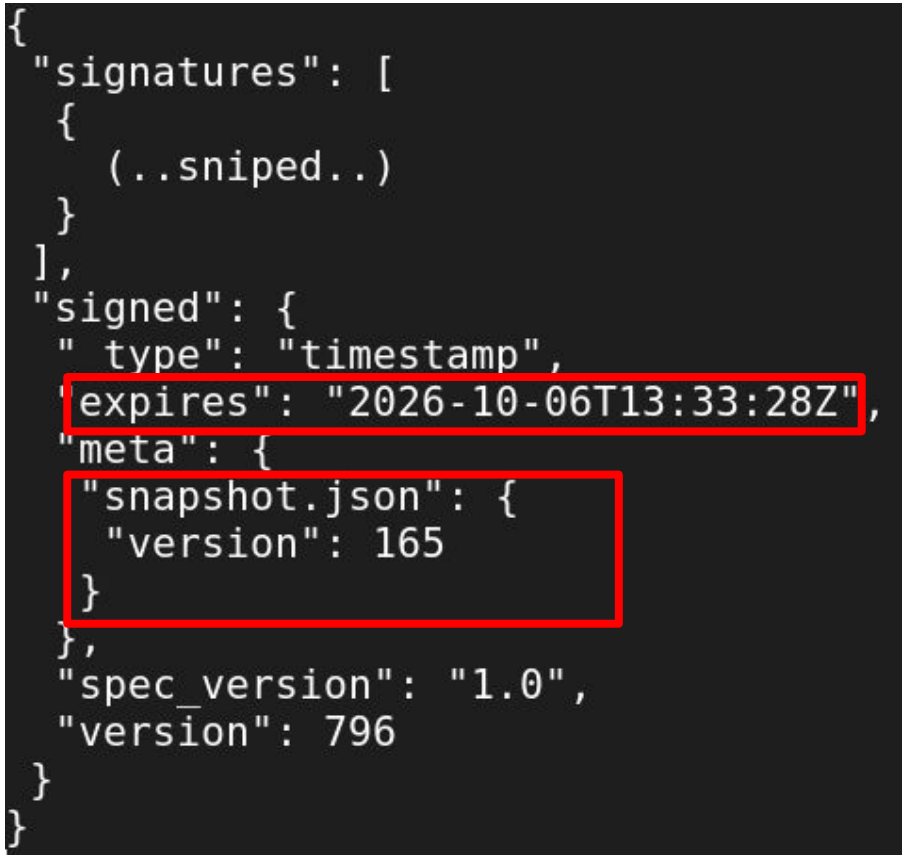
snapshot.jsonの例



```
{
 "signatures": [
  {
    (..sniped..)
  }
 ],
 "signed": {
  "_type": "snapshot",
  "expires": "2036-05-15T08:09:16Z",
  "meta": {
   "root.json": {
    "hashes": {
     "sha256": "f5ad89…",
     "sha512": "7445ddf…"
    },
    "length": 5297,
    "version": 2
   },
   "targets.json": {
    "version": 14
   }
  },
  "spec_version": "1.0",
  "version": 165
 }
}
```

- root.jsonとtargets.jsonのバージョンとハッシュ値(任意)が記録されている
- root.jsonとtargets.jsonのバージョンの組の文字通りスナップショット
- 特定のメタデータのみのすり替えを防ぐことができる(=メタデータの整合性)

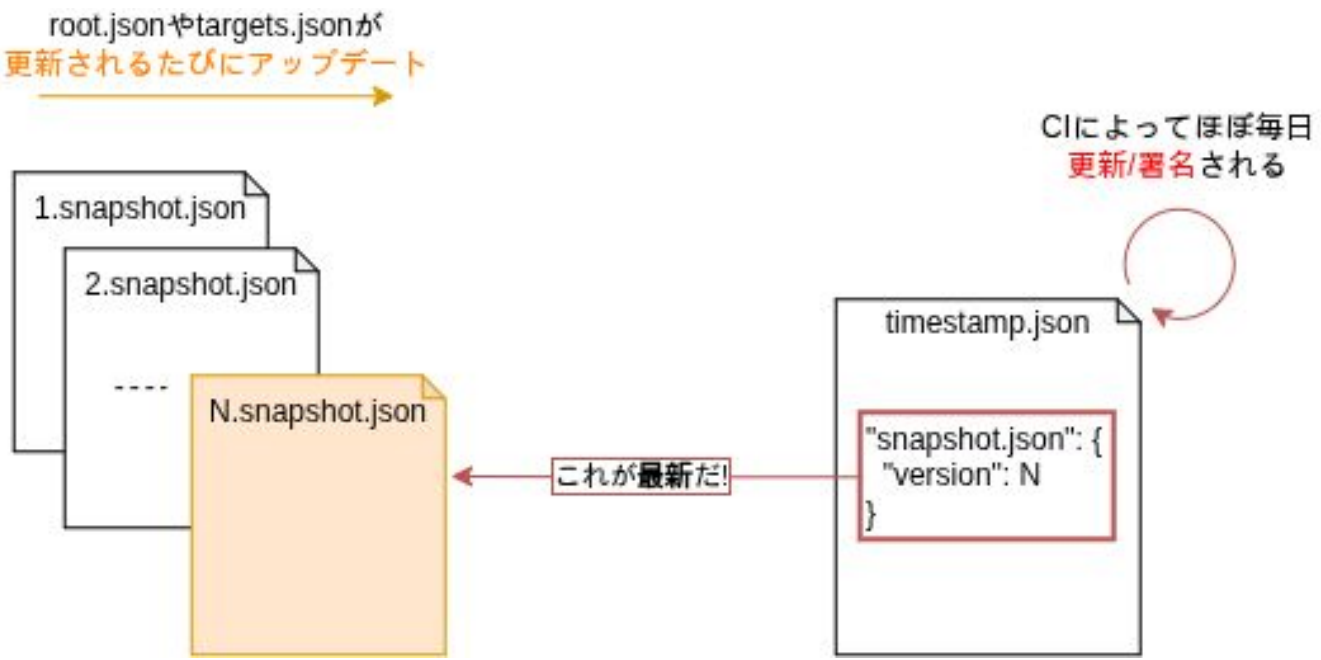
timestamp.jsonの例



```
{
 "signatures": [
  {
    (..sniped..)
  }
 ],
 "signed": {
  " type": "timestamp",
  "expires": "2026-10-06T13:33:28Z",
  "meta": {
   "snapshot.json": {
    "version": 165
   }
  },
  "spec_version": "1.0",
  "version": 796
 }
}
```

- snapshot.jsonのバージョンを示している
  - ここで示されるsnapshot.jsonが**最新**であることを表す
- 期限(expires)が非常に短く設定される
  - CIなどにより毎日更新されるような運用が想定されている
- 古いバージョンを最新と偽る攻撃(フリーズ攻撃)を防ぐ

メタデータの整合性と最新性



demo

Copyright Cybertrust Japan Co., Ltd. All rights reserved.

公開

demo

- ~~demo1: TUFのdemoリポジトリ~~
- demo2: sigstoreのTUFリポジトリ

demo2: sigstoreのTUFリポジトリ

sigstoreのTUFリポジトリ: https://tuf-repo-cdn.sigstore.dev/

↑のgitリポジトリ: https://github.com/sigstore/root-signing

ユーザーがインストールして実行するconsingのバイナリの署名の公開鍵が配布されており，バイナリを検証できる
(https://docs.sigstore.dev/cosign/system_config/installation/#verifying-cosign-with-artifact-key)

cosignの動作時には，Fulcio/Recorなどのtrust rootをTUFリポジトリから取得する
(https://oneuptime.com/blog/post/2026-08-26-distribute-private-fulcio-trust-root-tuf/view)

cybertrust

すべてのヒト、モノ、コトに 信頼を

留意事項

本資料に記載されている会社名、製品名、サービス名は、当社または各社、各団体の商標もしくは登録商標です。
その他本資料に記載されているイラスト・ロゴ・写真・動画・ソフトウェア等は、当社または第三者が有する知的財産権やその他の権利により守られております。
お客様は、当社が著作権を有するコンテンツについて、特に定めた場合を除き、複製、改変、頒布などをすることはできません。
本資料に記載されている情報は予告なしに変更されることがあります。また、時間の経過などにより記載内容が不正確となる場合がありますが、当社は、当該情報を更新する義務を負うものではありません。

Copyright Cybertrust Japan Co., Ltd. All rights reserved.